

Typing Simulator Report OOP

Typist Class

Encapsulation is where you hide the implementation of the code as much as possible, so there's restrictions to accessing those fields, e.g. all the fields declared in the class Typist are private (access modifier), which means other classes cannot change the game values directly (accuracy, progress, burnoutTurnsRemaining). This makes sure the object can't enter any invalid states such as progress going below zero, or accuracy outside the range of 0 to 1. Encapsulation allows code refactoring easily because when modifying code, other parts of the program aren't affected. It also supports abstraction.

Getters(accessors) are methods that 'get' a value (access and return the field) of the object, without actually altering the object or its fields. The getters included in the Typist class are: getName(), getSymbol(), getProgress(), getAccuracy(), getBurnoutTurnsRemaining(), isBurntOut().

Setters(mutators) are methods that 'set' an object's field to a specific value. They alter the state of the object. The setters in the Typist class are: setSymbol(char newSymbol), setAccuracy(double newAccuracy), typeCharacter(), slideBack(int amount), burnOut(int turns), recoverFromBurnout(), resetToStart().

The slideBack() method reverses the typist's progress if they make a mistake. The validation ensures that progress doesn't become negative. I do this by checking if progress is less than zero after the slide-back amount has been deducted and if it is, I set the progress to its minimum, which is zero. This ensures that the typist can't move before the actual passage starts, making sure all positions of the typist are valid and within the race/passage.

The setAccuracy() method modifies the typist's accuracy rating, ensuring it stays between the range of 0.0 and 1.0. If the input is below zero, the accuracy rating is set to zero. Likewise, if it's above one, the accuracy rating is set to one. Valid values are stored as they are inputted. This is important because typing success, mistypes and chance of burnout all depend on the accuracy rating.

```
J TypistTest.java X
Part1 > J TypistTest.java > ...
2 public class TypistTest {
4     public static void main(String[] args) {
5         Typist test = new Typist("E", "Test Typist", 0.75);
6
7         // Test 1: typeCharacter
8         test.typeCharacter();
9         test.typeCharacter();
10        test.typeCharacter();
11        System.out.println("Progress after 3 characters should be 3. Current Progress: " + test.getProgress());
12
13        // Test 2: slideBack shouldn't be less than zero
14        test.slideBack(5);
15        System.out.println("Progress after sliding back 5 should be 0. Current Progress: " + test.getProgress());
16
17        // Test 3: countdown for burnout
18        test.burnOut(3);
19        System.out.println("Is typist burnt out? Should be true. Burnt out: " + test.isBurntOut());
20        System.out.println("Burnout turns remaining should be 3. Current burnout turns remaining: " + test.getBurnoutTurnsRemaining());
21
22        test.recoverFromBurnout();
23        System.out.println("After 1 recovery, burnout turns remaining should be 2. Current burnout turns remaining: " + test.getBurnoutTurnsRemaining());
24        test.recoverFromBurnout();
25        System.out.println("After 2 recoveries, burnout turns remaining should be 1. Current burnout turns remaining: " + test.getBurnoutTurnsRemaining());
26        test.recoverFromBurnout();
27        System.out.println("After 3 recoveries, typist should no longer be burnt out. Burnt out: " + test.isBurntOut());
28        System.out.println("Burnout turns remaining should be 0. Current burnout turns remaining: " + test.getBurnoutTurnsRemaining());
29
30        // Test 4: testing accuracy in range
31        test.setAccuracy(1.2);
32        System.out.println("Accuracy should be capped at 1.0. Current accuracy: " + test.getAccuracy());
33        test.setAccuracy(-0.7);
34        System.out.println("Accuracy should be floored at 0.0. Current accuracy: " + test.getAccuracy());
35        test.setAccuracy(0.5);
36        System.out.println("Accuracy should be set to 0.5. Current accuracy: " + test.getAccuracy());
37
38        // Test 5: resetToStart()
39        test.typeCharacter();
40        test.typeCharacter();
41        test.burnOut(2);
42        test.resetToStart();
43
44        System.out.println("After reset, progress should be 0. Current Progress: " + test.getProgress());
45        System.out.println("After reset, typist should not be burnt out. Burnt out: " + test.isBurntOut());
46        System.out.println("After reset, burnout turns remaining should be 0. Current burnout turns remaining: " + test.getBurnoutTurnsRemaining());
47    }
48 }
49 }

C:\Users\Umerl\OneDrive\Desktop\OOP coursework\TypingRaceSimulator\TypingRaceSimulator\Part1>javac Typist.java TypistTest.java
C:\Users\Umerl\OneDrive\Desktop\OOP coursework\TypingRaceSimulator\TypingRaceSimulator\Part1>java TypistTest
Progress after 3 characters should be 3. Current Progress: 3
Progress after sliding back 5 should be 0. Current Progress: 0
Is typist burnt out? Should be true. Burnt out: true
Burnout turns remaining should be 3. Current burnout turns remaining: 3
After 1 recovery, burnout turns remaining should be 2. Current burnout turns remaining: 2
After 2 recoveries, burnout turns remaining should be 1. Current burnout turns remaining: 1
After 3 recoveries, typist should no longer be burnt out. Burnt out: false
Burnout turns remaining should be 0. Current burnout turns remaining: 0
Accuracy should be capped at 1.0. Current accuracy: 1.0
Accuracy should be floored at 0.0. Current accuracy: 0.0
Accuracy should be set to 0.5. Current accuracy: 0.5
After reset, progress should be 0. Current Progress: 0
After reset, typist should not be burnt out. Burnt out: false
After reset, burnout turns remaining should be 0. Current burnout turns remaining: 0
```

I created a separate file called TypistTest.java containing a main method, with all the tests for the report in it.

typeCharacter() Test:

I called typeCharacter() for the object of class Typist called test 3 times, so I printed a line that said the progress was expected to be 3, since the typist advanced 3 characters. The test also confirmed the progress was 3 so typeCharacter() moves forward normally.

slideBack() Test:

I called slideBack() for the same object of class Typist called test, with the parameter of the integer 5 for the amount, but since the progress can't be negative as explained before (otherwise it would've been -2), the progress was outputted as zero (matching the line I printed to say the progress should be 0), confirming the progress can't go below zero after calling slideBack().

burnout() Test:

I called burnOut() for the same object of class Typist called test, with the parameter of the integer 3 for the turns. The expected behaviour was that the Boolean burntOut should be assigned the value of true and the burnoutTurnsRemaining should be equal to the parameter turns inputted. The burnout Boolean was outputted as true and the burnoutTurnsRemaining mirrored the integer inputted, matching expectations.

setAccuracy() Test:

I called setAccuracy() for the same object of class Typist called test, with the parameter of the doubles 1.2, -0.7 and 0.5 for the accuracy rating. The expected results were that the accuracy rating stays between the range of 0.0 and 1.0. If the accuracy rating is below zero (-0.7), it is set to zero. If it's above one (1.2), the accuracy rating is set to one. Valid values (0.5) are unchanged.

resetToStart() Test:

I called resetToStart() for the same object of class Typist called test. The expected behaviour after the reset was, the progress should be set to 0, the typist should not be burnt out and thus, the burnout turns remaining should be 0. The outputs were: the current progress is 0, the Boolean isBurntOut returned false and the current burnout turns remaining were 0, so the progress and burnout state were both cleared.

TypingRace Class Report:

After looking through the original TypingRace code, I found 7 issues. These were all issues that didn't allow the race to work correctly. I fixed these one by one to make sure the race runs correctly.

The first issue was that not all typists were being reset at the start of the race. The original code reset seat1Typist and seat2Typist, but not seat3Typist. This made it unfair because the third typist might have had leftover progress or burnout from a previous race. I fixed this in startRace() by calling resetToStart() for seat3Typist too. The correct implementation should make sure every typist starts from the same initial state.

Code: seat3Typist.resetToStart();

The second issue was that there were no null checks for typists. In the original code, it was assumed all three seats always had a typist. If a seat was empty, this could cause a `NullPointerException` crash when methods like `resetToStart()`, `advanceTypist()`, `raceFinishedBy()`, or `printSeat()` were called. I fixed this by checking whether the typist is null before using it. The correct implementation should ignore empty seats instead of crashing.

```
Code: if (seat1Typist != null) {  
    seat1Typist.resetToStart();  
}  
if (seat2Typist != null) {  
    seat2Typist.resetToStart();  
}  
if (seat3Typist != null) {  
    seat3Typist.resetToStart();  
}
```

The third problem was with the mistype probability in `advanceTypist()`. Originally, more accurate typists were more likely to mistype, which is unfair and counterintuitive. I changed it so the chance of mistyping depends on $(1 - \text{accuracy})$. So now, higher accuracy means a lower chance of making a mistake.

```
Code: if (Math.random() < (1 - theTypist.getAccuracy()))
```

The race finish condition in `raceFinishedBy()` was also incorrect. The original code checked whether progress was exactly equal to `passageLength`. This is wrong if a typist's progress went past the target. For example, if the passage length was 40 and the typist reached 41, they should still win. I fixed this by using `>=` instead of `==`. The correct implementation should declare a winner when progress is greater than or equal to the passage length.

```
Code: if (theTypist.getProgress() >= passageLength)
```

Another issue was that the winner was not printed at the end of the race. This is not good because the race ended but did not clearly show the winner. I fixed this by storing

the winning typist in a winner variable and printing their name and final accuracy. The correct implementation should output the winner once the race finishes.

```
Code: System.out.println(" 🏁 " + winner.getName() + " wins the race!");
```

There were also issues in printSeat(). If a typist's progress went past the passage length, the spacing could become incorrect and affect the race picture. I fixed this by keeping the displayed progress as much as the passage length before calculating the spaces. The correct implementation should keep the visual output readable even if the progress overshoots.

```
Code: if (progress > passageLength) {  
    progress = passageLength;  
}
```

Finally, when a typist was burnt out, the extra ~ symbol could cause spacing issues. I adjusted the spacing so that it does not become negative. The correct implementation should show burnout clearly without breaking the formatting.

```
Code: if (theTypist.isBurntOut()) {  
    System.out.print('~');  
    if (spacesAfter > 0) {  
        spacesAfter--; // ~ takes one character space  
    }  
}
```

Optionally, I added a small improvement to show when a typist has just mistyped. I use a temporary flag in the Typist class, which is set when slideBack() is called and cleared after being displayed. The typist is shown with a < symbol for one turn, making the race output clearer.

Overall, these fixes made the race correct and fair. I also fixed the readability of the race's picture. Now, it handles completing the race, empty seats, mistypes, burnout display, and winner output properly.